



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **state space** is a property of the environment. The set of all distinct configurations plus the transitions between them, forming a graph in which each state appears exactly **once**. In my implementation a state is just the coordinate (x, y), so the 12×12 grid has 144 cells minus 5 walls = 139 reachable states, with edges wherever `get_successors` permits a legal move. The **search tree** is a property of the algorithm. A runtime record whose nodes are not states but **paths** from the start, so the same state can appear as many different nodes. For example (0,0) is one single state, yet it appears in the tree as the root, again via Right --> Left, and again via Up --> Down.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It solves the **redundant path / cycle problem**, converting *tree search* into *graph search*, which matters here because a grid is saturated with cycles. Every Up is undone by a Down. For **DFS the consequence is fatal rather than merely slow**: from (0,0) it pushes (1,0), from (1,0) it pushes (0,0) again as a *deeper* node, and since the LIFO stack always returns the newest node it descends this two-cell oscillation forever, never terminating even when the goal is two cells away. This is precisely the difference between DFS being *complete* and *incomplete*. With `reached` it is complete on a finite state space, without it it is not.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS is optimal here because of one specific property: **every action costs exactly 1**, so path cost and path depth are the same quantity, and BFS's FIFO queue exhausts every path of length k before touching any of length $k+1$. Meaning the first time the goal is popped, no shorter path can exist. DFS produces winding routes because it returns the **first** path it stumbles upon and has no mechanism to compare two paths; its LIFO stack drives one branch to maximum depth, and my `ACTIONS` ordering makes `pop()` yield Right first, so it serpentes across nearly the whole board.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

By the lecture formulas, BFS is $O(b^d)$ time and **$O(b^d)$ space** because it is *breadth-resident*. It must hold an entire frontier level plus the `reached` set at once, whereas DFS is $O(b^m)$ time but only **$O(b \cdot m)$ space**, being *depth-resident* and storing just the current path and its siblings.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING